

TRAINING SIMPLE MACHINE LEARNING ALGORITHMS

Perceptron, Adaline & Gradient Descent

From rule-based updates to continuous optimization of a cost function

Dr. Md. Shahidur Rahman
Professor, DoCSE, SUST
rahmanms@sust.edu

Roadmap

1

Perceptron & Adaline

Why a continuous activation function changes everything

2

The Cost Function

Sum of squared errors as an optimization objective

3

Gradient Descent

Climbing down the cost surface, one step at a time

4

Feature Scaling

Standardization speeds up convergence

5

SGD & Mini-Batch

Scaling learning to large, streaming datasets

ORIGINS

A brief history of the artificial neuron

1943

McCulloch & Pitts

Proposed the McCulloch–Pitts (MCP) neuron: a simplified model of a brain cell, aiming to understand how biological neurons could inspire artificial intelligence.

Today
Foundations

These ideas underpin every modern classifier: a weighted sum of inputs compared against a threshold.

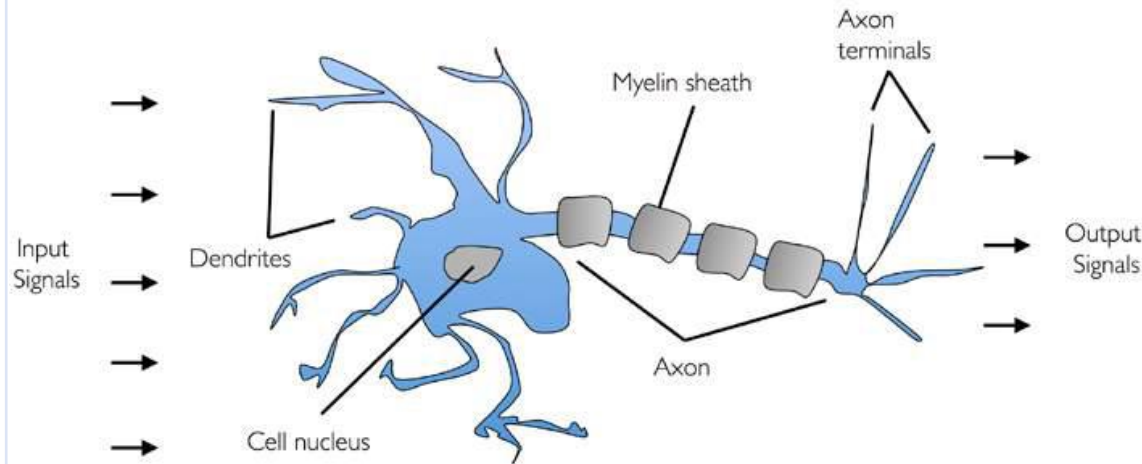
1957

Frank Rosenblatt

Published the perceptron learning rule, building on the MCP model — the first algorithm that could automatically learn optimal weights from data.

From biological neuron to logic gate

Biological neuron



Multiple signals arrive at the dendrites and are integrated in the **cell body**. When the accumulated signal exceeds a threshold, the **neuron fires** and passes a signal along the axon.

The MCP reframing

McCulloch and Pitts described this behavior in computational terms:

- A simple logic gate with binary output
- Inputs are integrated (summed) in the cell body
- An output fires only if the accumulated signal exceeds a threshold

This binary, threshold-based logic gate is the seed from which the perceptron grows.

The net input: a weighted sum

For an input vector x and weight vector w , the perceptron computes a net input z as their dot product:

$$z = w_0x_0 + w_1x_1 + \dots + w_mx_m = w^T x$$

$x_0 = 1$ is a constant input tied to the bias weight w_0 (the negative threshold).

Worked example

$$[1 \ 2 \ 3] \times [4 \ 5 \ 6]^T = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

Decision rule

Compare z to a threshold θ :

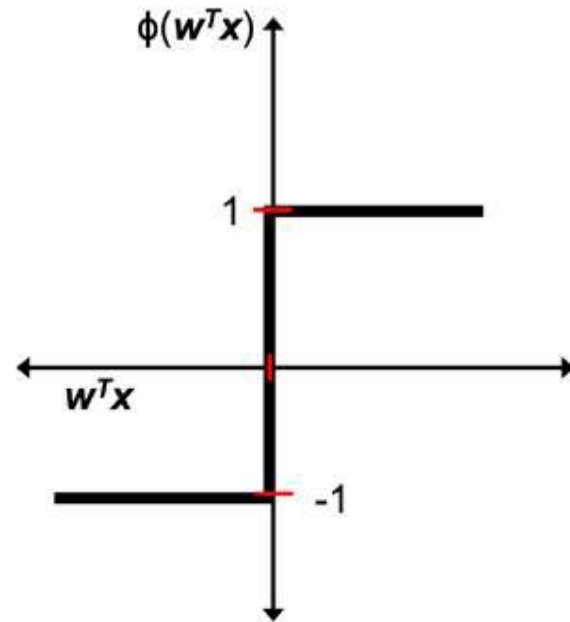
$$z \geq \theta \rightarrow \text{predict class +1}$$

$$z < \theta \rightarrow \text{predict class -1}$$

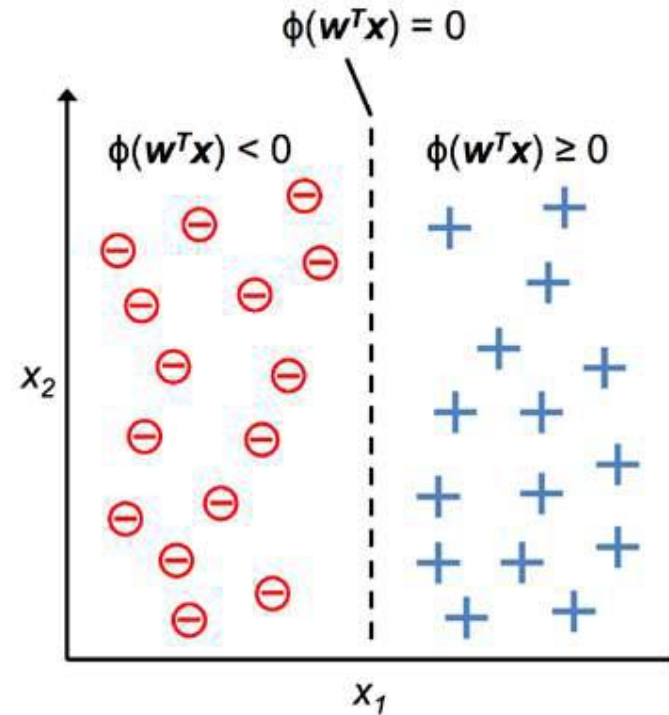
Folding $-\theta$ into w_0 lets us write the rule compactly as $z \geq 0$ vs. $z < 0$. The value $w_0 = -\theta$ is called the **bias unit**.

Squashing z into a binary output

The unit step function $\phi(w^T x)$



Discriminating two linearly separable classes

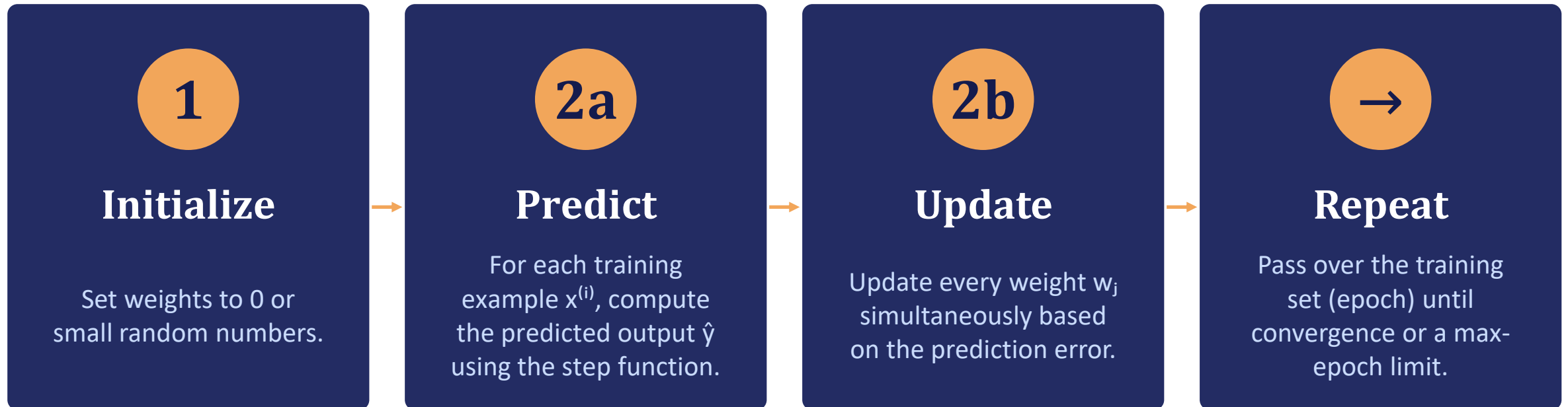


If the accumulated net input meets the threshold, the neuron outputs +1; otherwise, it outputs -1.

THE ALGORITHM

The perceptron learning rule

A reductionist approach mimicking a single neuron: it either fires, or it doesn't.



THE ALGORITHM

Updating the weights

Each weight is updated simultaneously as:

$$w_j := w_j + \Delta w_j$$

$$\Delta w_j = \eta (y^{(i)} - \hat{y}^{(i)}) x_j^{(i)}$$

η learning rate, typically between 0.0 and 1.0

$y^{(i)}$ the true class label of example i

$\hat{y}^{(i)}$ the predicted class label of example i

Two quick examples

Misclassified as -1 , with $x_j^{(i)} = 0.5$:

$$\Delta w_j = (1 - (-1))(0.5) = 1$$

Misclassified as -1 , with a larger $x_j^{(i)} = 2$:

$$\Delta w_j = (1 - (-1))(2) = 4$$

Larger input values push the decision boundary by a proportionally larger amount — the update scales with $x_j^{(i)}$.

When do weights actually change?

Correct

True label +1,
predicted +1

$$\Delta w_j = \eta(1-1)x = 0$$

No update — the prediction was already right.

Correct

True label -1,
predicted -1

$$\Delta w_j = \eta(-1-(-1))x = 0$$

No update — the prediction was already right.

Wrong

True label +1,
predicted -1

$$\Delta w_j = \eta(1-(-1))x = 2\eta x$$

Weights pushed toward the positive class.

Wrong

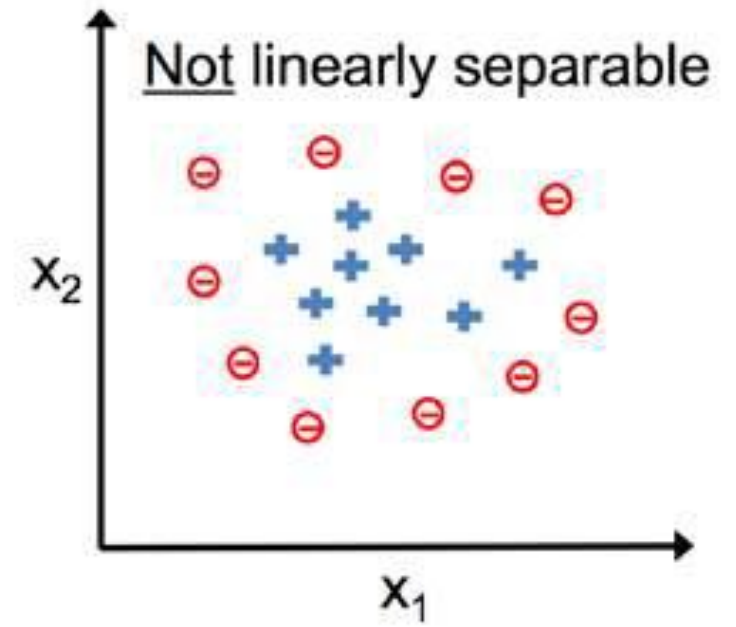
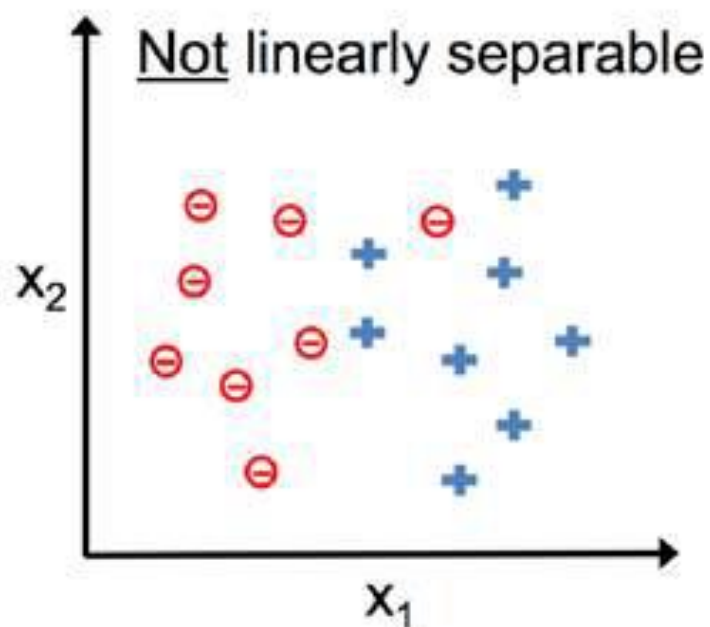
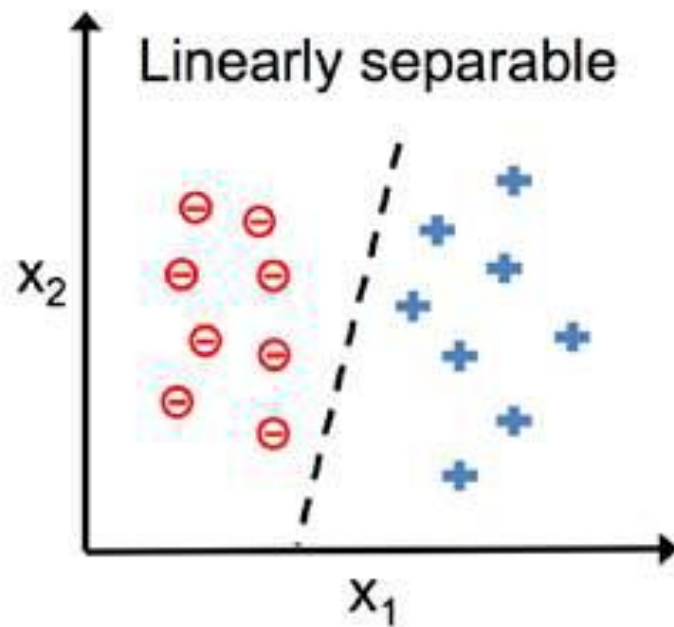
True label -1,
predicted +1

$$\Delta w_j = \eta(-1-1)x = -2\eta x$$

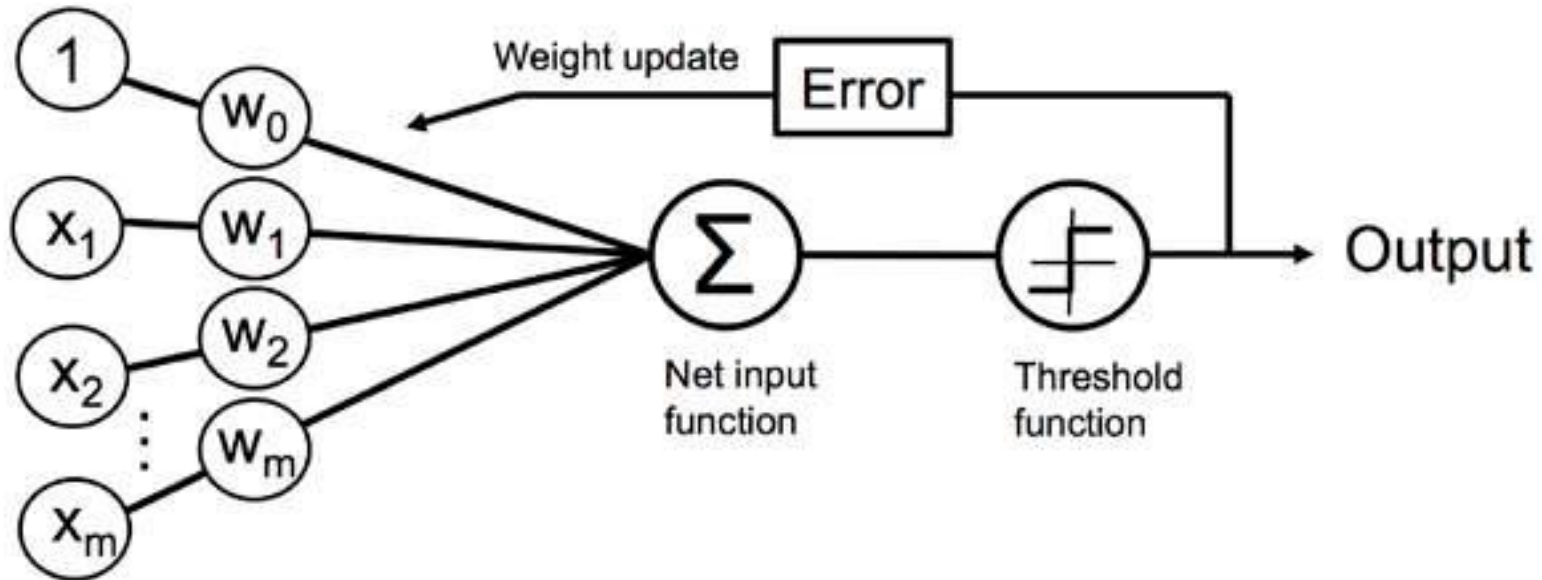
Weights pushed toward the negative class.

Convergence depends on separability

Convergence is only guaranteed if the two classes are **linearly separable** and the learning rate is small enough. Otherwise, the perceptron will never stop updating — so we cap training with a maximum number of epochs and/or a tolerated error threshold.



The perceptron, end to end



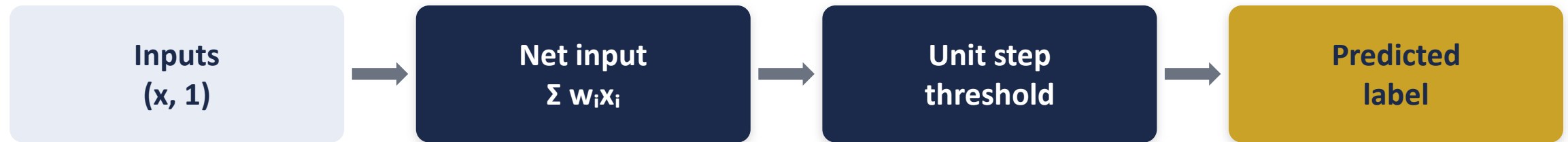
Inputs are combined with weights into a net input, passed through a threshold function, and turned into a binary prediction. During learning, the **resulting error flows back to update every weight.**

SECTION 1

Perceptron vs. Adaline

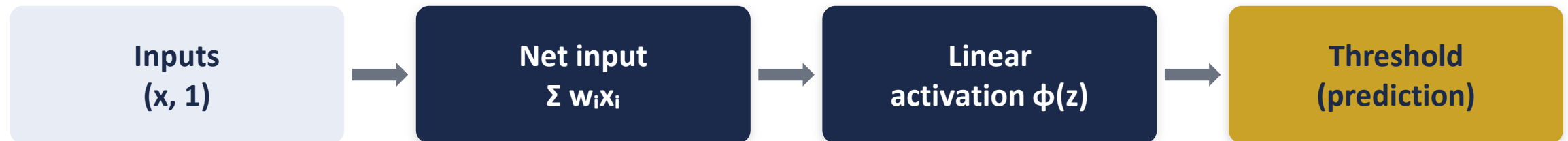
Same inputs, same weighted sum — a different signal drives the weight update.

PERCEPTRON



Weights update using the PREDICTED CLASS LABEL vs. the true label (integer comparison).

ADALINE



Weights update using (true label – $\phi(z)$) — the CONTINUOUS activation output, before thresholding.

SECTION 2

Defining the Cost Function

Every supervised learner needs an **objective function** to optimize during training.

- Adaline defines cost $J(w)$ as **the Sum of Squared Errors (SSE)** between the true label and the continuous output $\phi(z)$
- The $\frac{1}{2}$ term is a convenience factor — it cancels cleanly when we differentiate
- Because $\phi(z)$ is a linear (identity) function, $J(w)$ is differentiable and convex
- A convex, differentiable cost function guarantees gradient descent can find the global minimum

COST FUNCTION

$$J(w) = \frac{1}{2} \sum_i (y^{(i)} - \phi(z^{(i)}))^2$$

$y^{(i)}$ true class label for example i

$z^{(i)}$ net input, $w^T x^{(i)}$

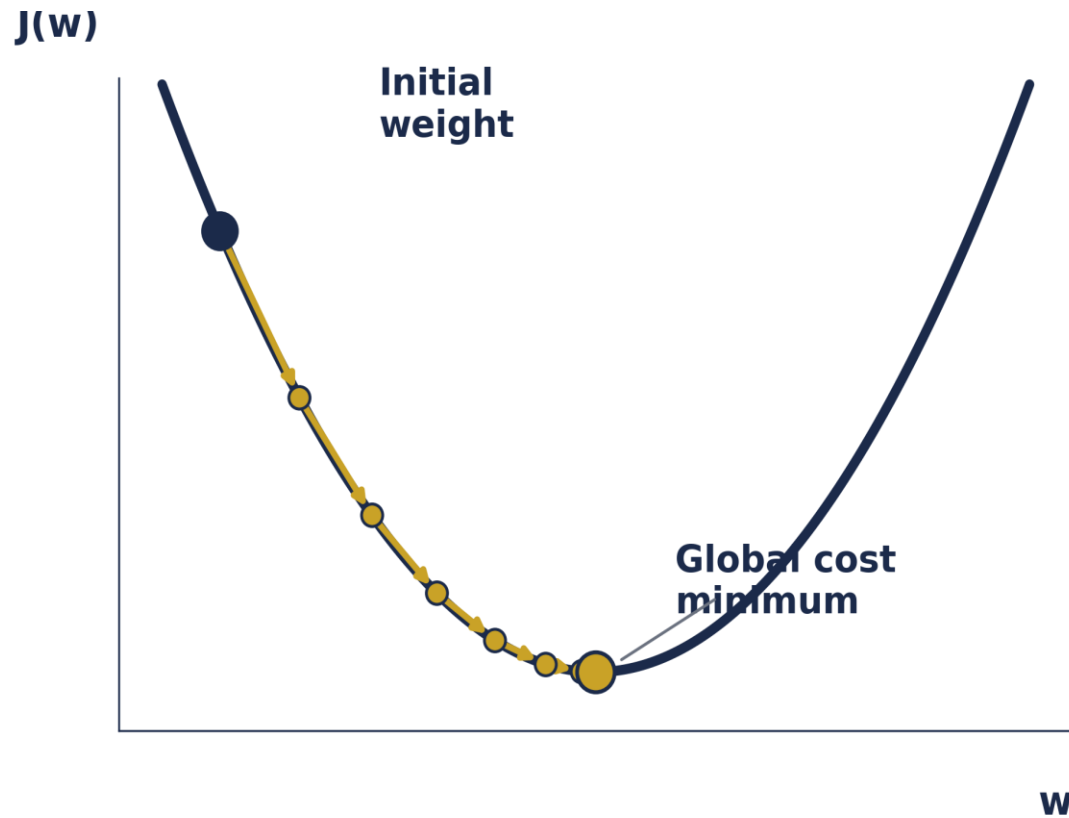
$\phi(z)$ linear activation, $\phi(z) = z$

Why not just count misclassifications?

A 0/1 misclassification count is flat almost everywhere — its gradient is zero, so there is no signal to guide learning. SSE is smooth instead.

SECTION 3

Gradient Descent: Climbing Down the Hill



The step-by-step idea

- 1 Start from an initial (often random) weight vector w
- 2 Compute the gradient $\nabla J(w)$ — the slope of the cost surface at the current point
- 3 Take a step in the OPPOSITE direction of the gradient
- 4 Step size is controlled by the learning rate η
- 5 Repeat until the cost stops decreasing meaningfully

$$w := w + \Delta w \quad \Delta w = -\eta \nabla J(w)$$

SECTION 3

Deriving the Weight Update Rule

Partial derivative of cost w.r.t. w_j $\partial J / \partial w_j = - \sum_i (y^{(i)} - \phi(z^{(i)})) x_j^{(i)}$

Weight change for w_j $\Delta w_j = -\eta (\partial J / \partial w_j) = \eta \sum_i (y^{(i)} - \phi(z^{(i)})) x_j^{(i)}$

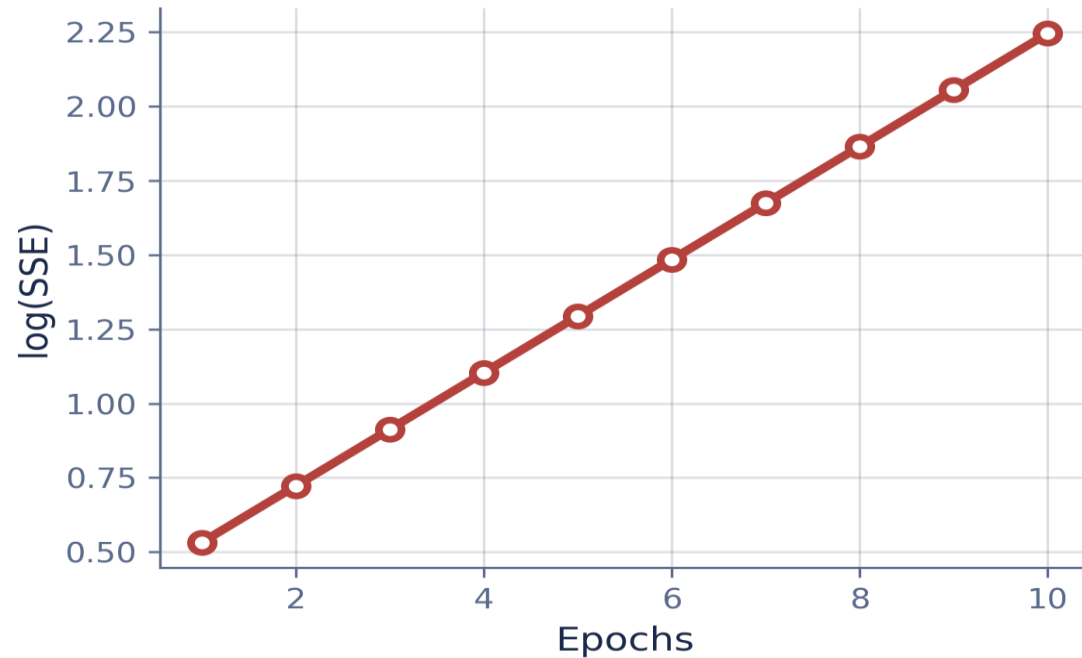
Simultaneous update, all weights $\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}$

Key distinction from the perceptron: $\phi(z^{(i)})$ here is a real-valued number, not an integer class label — and the update sums over the *entire training set* before any weight changes, which is why this is called **batch gradient descent**.

SECTION 3

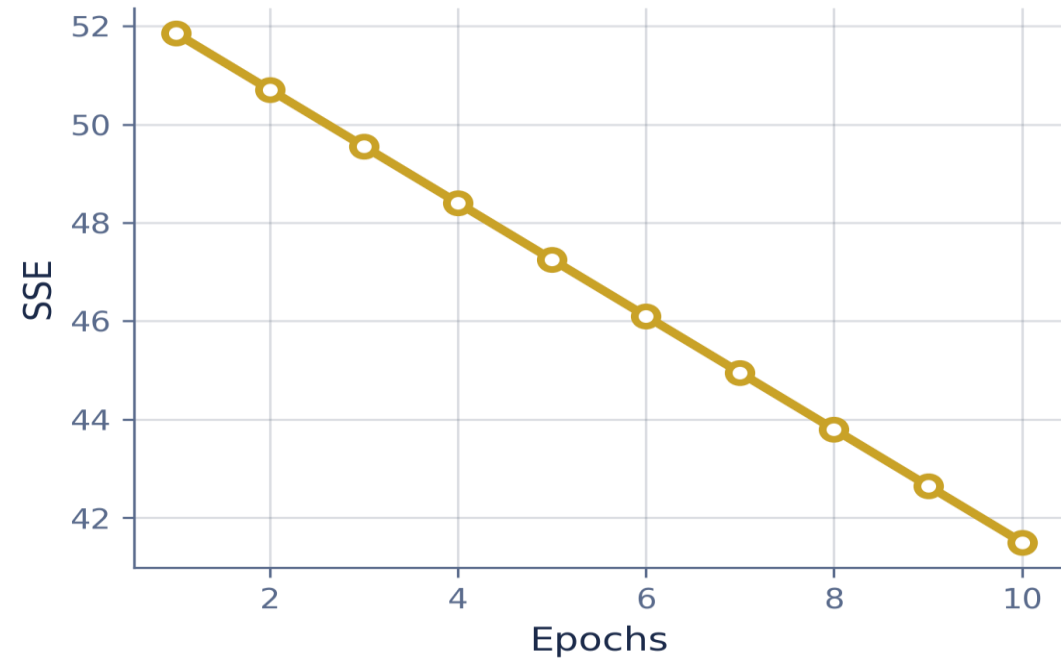
Choosing the Learning Rate Matters

Learning rate too large ($\eta = 0.01$)



Too large $\rightarrow$ overshoots the minimum; cost grows every epoch instead of shrinking.

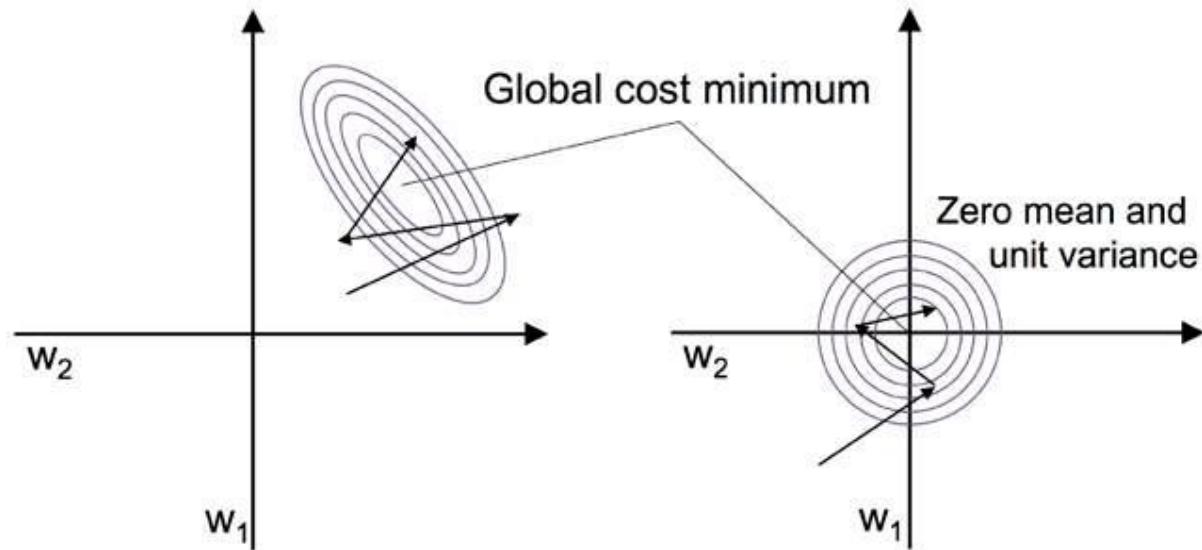
Learning rate too small ($\eta = 0.0001$)



Too small $\rightarrow$ converges safely, but needs far too many epochs to be practical.

SECTION 4

Improving Convergence with Feature Scaling



STANDARDIZATION

$$\mathbf{x}'_j = (\mathbf{x}_j - \mu_j) / \sigma_j$$

Zero-mean, unit-variance features reshape the cost surface into rounder, more symmetric contours.

Result: gradient descent takes fewer, more direct steps to reach the minimum.

In NumPy: `X_std[:, j] = (X[:, j] - X[:, j].mean()) / X[:, j].std()`

*Note: standardization does not make the data normally distributed — it **only recenters and rescales each feature**. After scaling, Adaline converges within about 15 epochs at $\eta = 0.01$, and every example is classified correctly — though SSE never quite reaches zero.*

SECTION 5

Batch Gradient Descent Doesn't Scale

Every single weight update re-evaluates the ENTIRE training set — fine for the 100-row Iris dataset, but costly once we reach millions of examples.

Batch GD

$$\Delta \mathbf{w} = \eta \sum_i (\mathbf{y}^{(i)} - \phi(\mathbf{z}^{(i)})) \mathbf{x}^{(i)}$$

One update per epoch, using all m examples
— stable but slow at scale.

Stochastic GD

$$\Delta \mathbf{w} = \eta (\mathbf{y}^{(i)} - \phi(\mathbf{z}^{(i)})) \mathbf{x}^{(i)}$$

One update per training example — noisy but fast, and enables online learning.

Both share the same underlying idea — the difference is how much data each weight update looks at before moving.

SECTION 5

Mini-Batch: The Practical Middle Ground

Applies batch gradient descent to small subsets of the data (e.g., 32 examples at a time) — combining faster convergence with vectorized, efficient computation.

	Batch GD	Stochastic GD	Mini-Batch GD
Update frequency	Per epoch (all m)	Per example	Per small batch
Gradient noise	None — smooth	High — noisy	Moderate
Convergence speed	Slow per update	Fast, erratic path	Fast & efficient
Best suited for	Small datasets	Streaming / online	Large-scale training

For most modern deep learning workloads, mini-batch gradient descent is the default choice — it balances speed, stability, and hardware efficiency.

RECAP

Key Takeaways

- Adaline updates weights using a continuous linear activation, not the thresholded prediction — making its cost function differentiable
- SSE is a convex, differentiable cost function — the ideal setting for gradient descent
- The learning rate η trades off stability against speed of convergence
- Standardizing features reshapes the cost surface and speeds up convergence dramatically
- Batch, stochastic, and mini-batch gradient descent trade computation cost against update frequency and noise